

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №1
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Застосування принципів SOLID об'єктно-орієнтованого програмування при проєктуванні програмних систем у сфері телекомунікацій.

Мета роботи: Ознайомитися з принципами SOLID. Навчитися застосовувати їх під час проєктування класів на Python. Формувати навички створення гнучкого, масштабованого та підтримуваного коду.

Теоретичні відомості

SOLID – це набір із п'яти основних архітектурних принципів об'єктно-орієнтованого програмування. Їх використання дозволяє створювати масштабовані системи, які легко модифікувати та тестувати, що є критично важливим для телекомунікацій, де системи взаємодіють у реальному часі.

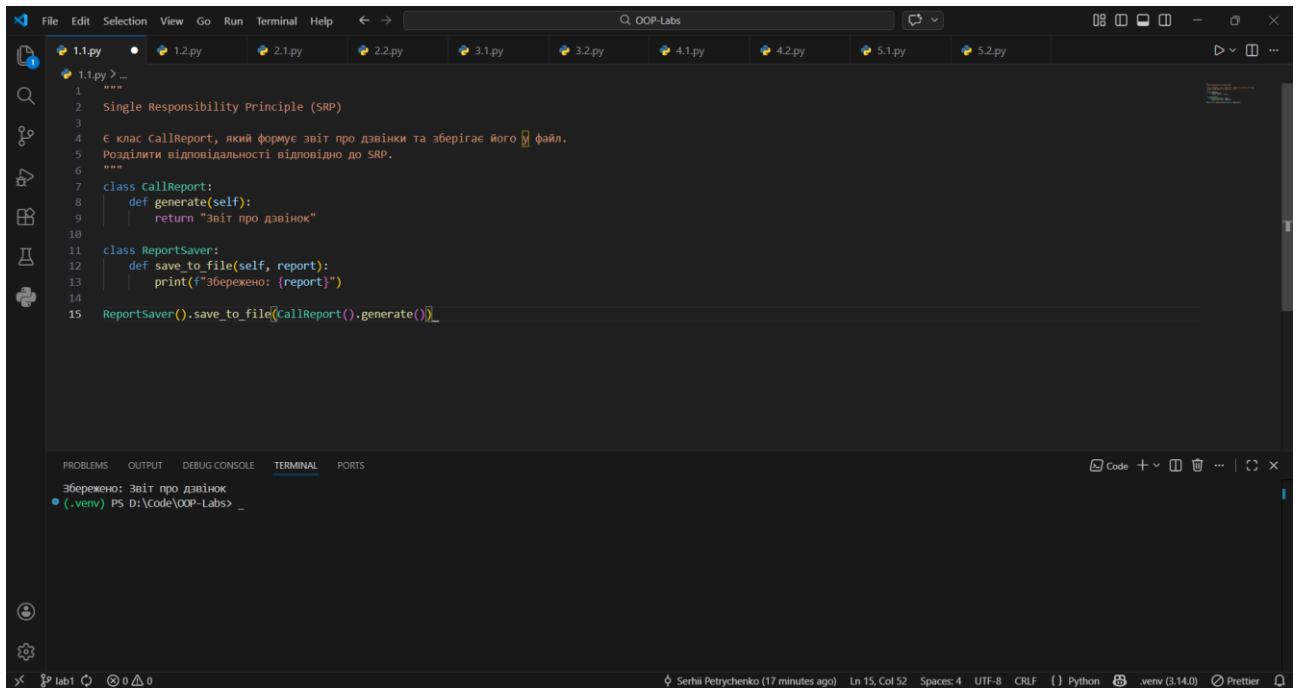
Принципи SOLID:

- S (Single Responsibility Principle): Клас повинен мати лише одну відповідальність і одну причину для зміни.
- O (Open/Closed Principle): Програмні сутності мають бути відкритими для розширення, але закритими для модифікації. Поведінка розширюється без зміни існуючого коду.
- L (Liskov Substitution Principle): Підкласи повинні замінювати базові класи без порушення логіки програми та змін у очікуваній поведінці.
- I (Interface Segregation Principle): Краще створювати багато спеціалізованих інтерфейсів, ніж один універсальний інтерфейс, що змушує реалізовувати зайві методи.
- D (Dependency Inversion Principle): Високорівневі модулі не повинні залежати від низькорівневих; обидва типи мають залежати від абстракцій.

Окрім SOLID, для побудови архітектури ТК-систем використовуються патерни проєктування, такі як Singleton (єдиний екземпляр для управління мережею), Factory Method (створення підключень) та Observer (оновлення статусу вузлів).

Результати виконаної роботи

Завдання 1.1



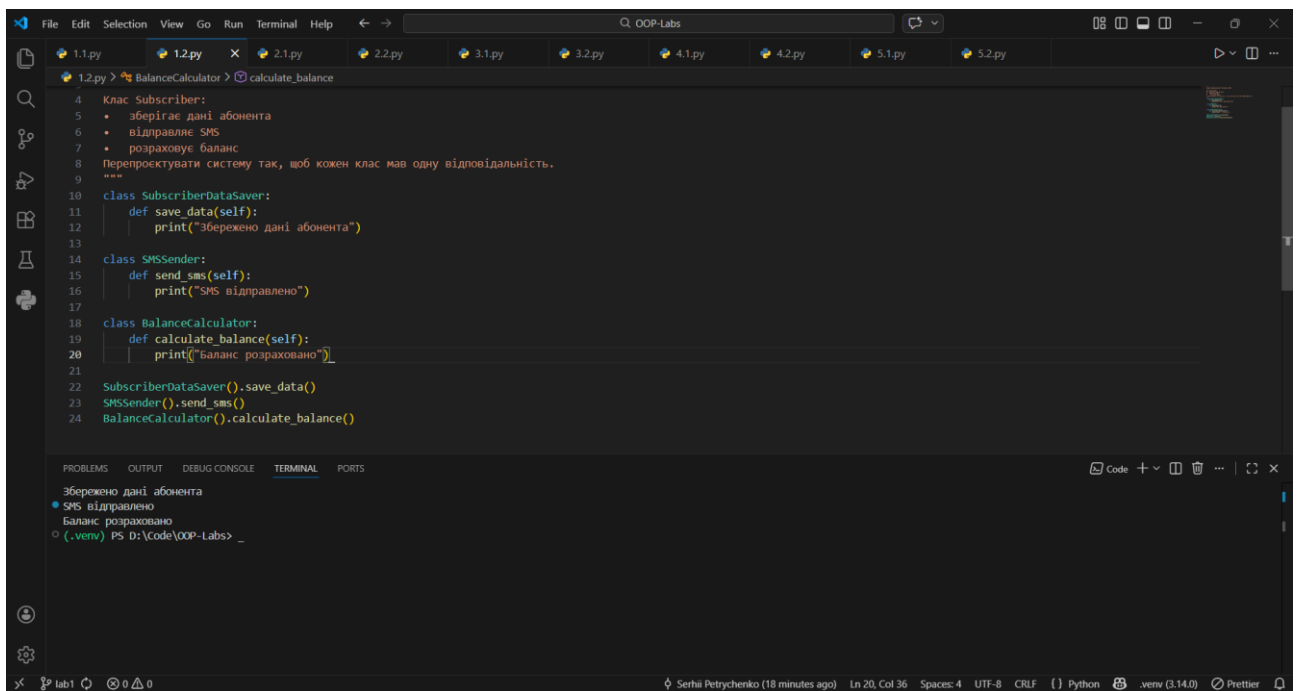
```
1.1.py > _
1  """
2  Single Responsibility Principle (SRP)
3
4  Є клас CallReport, який формує звіт про дзвінки та зберігає його у файл.
5  Розділити відповідальності відповідно до SRP.
6  """
7  class CallReport:
8      def generate(self):
9          return "Звіт про дзвінок"
10
11  class ReportSaver:
12      def save_to_file(self, report):
13          print(f"Збережено: {report}")
14
15  ReportSaver().save_to_file(CallReport().generate())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Збережено: Звіт про дзвінок
(.venv) PS D:\code\OOP-Labs> _

Скріншот 1 – Виконання завдання 1.1

Завдання 1.2



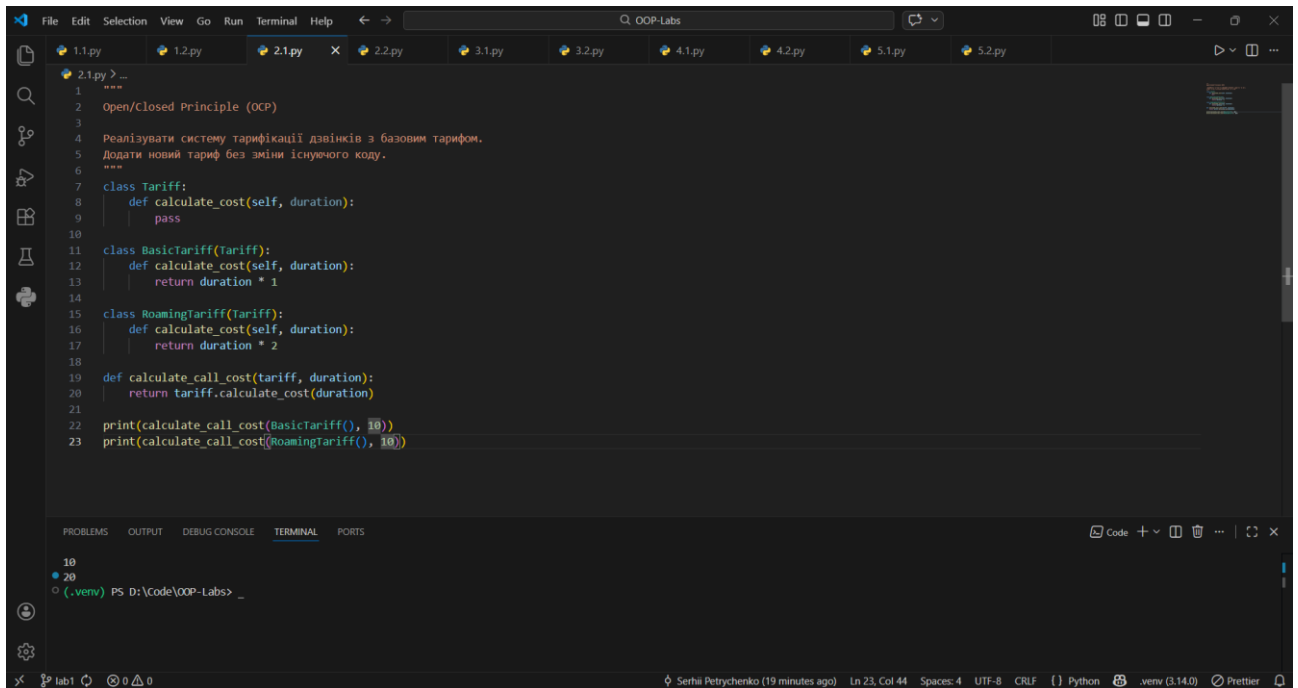
```
1.2.py > BalanceCalculator calculate_balance
4  Клас Subscriber:
5  • зберігає дані абонента
6  • відправляє SMS
7  • розраховує баланс
8  Перепроєктувати систему так, щоб кожен клас мав одну відповідальність.
9  """
10  class SubscriberDataSaver:
11      def save_data(self):
12          print("Збережено дані абонента")
13
14  class SMSender:
15      def send_sms(self):
16          print("SMS відправлено")
17
18  class BalanceCalculator:
19      def calculate_balance(self):
20          print("Баланс розраховано")
21
22  SubscriberDataSaver().save_data()
23  SMSender().send_sms()
24  BalanceCalculator().calculate_balance()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Збережено дані абонента
SMS відправлено
Баланс розраховано
(.venv) PS D:\code\OOP-Labs> _

Скріншот 2 – Виконання завдання 1.2

Завдання 2.1

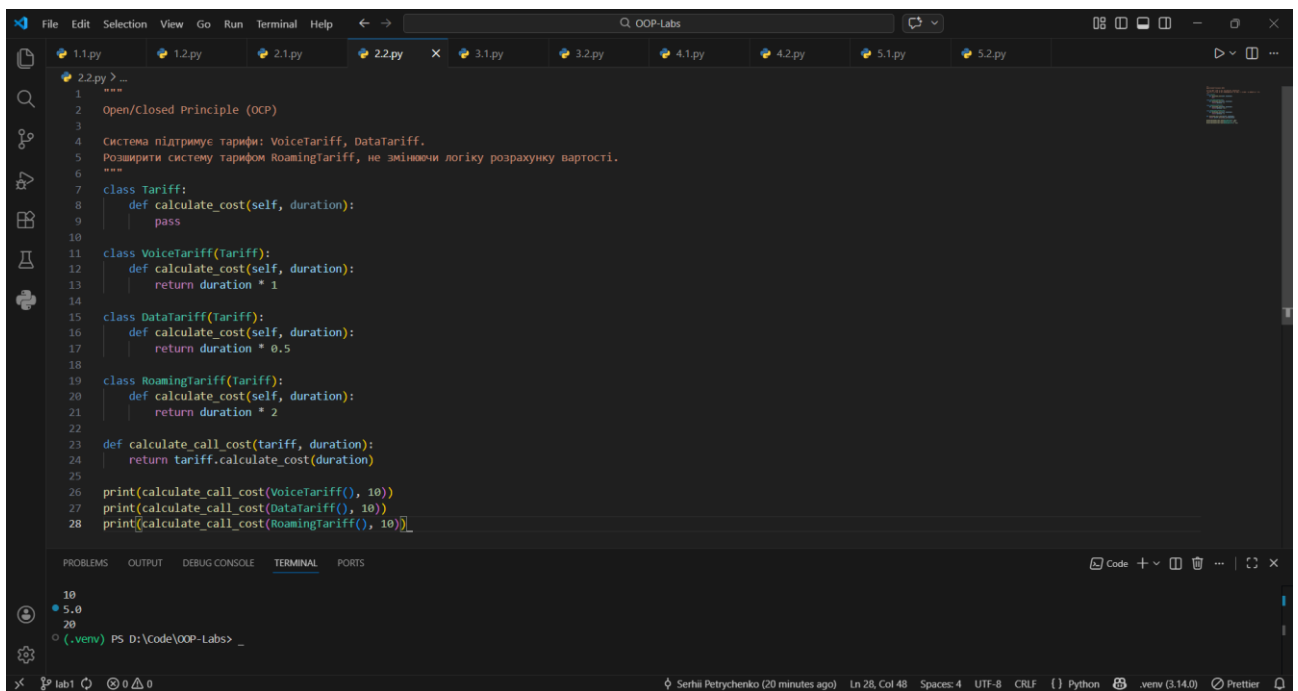


```
1 """
2 Open/Closed Principle (OCP)
3
4 Реалізувати систему тарифікації дзвінків з базовим тарифом.
5 Додати новий тариф без зміни існуючого коду.
6 """
7 class Tariff:
8     def calculate_cost(self, duration):
9         pass
10
11 class BasicTariff(Tariff):
12     def calculate_cost(self, duration):
13         return duration * 1
14
15 class RoamingTariff(Tariff):
16     def calculate_cost(self, duration):
17         return duration * 2
18
19 def calculate_call_cost(tariff, duration):
20     return tariff.calculate_cost(duration)
21
22 print(calculate_call_cost(BasicTariff(), 10))
23 print(calculate_call_cost(RoamingTariff(), 10))
```

10
20
PS D:\Code\OOP-Labs>

Скріншот 3 – Виконання завдання 2.1

Завдання 2.2

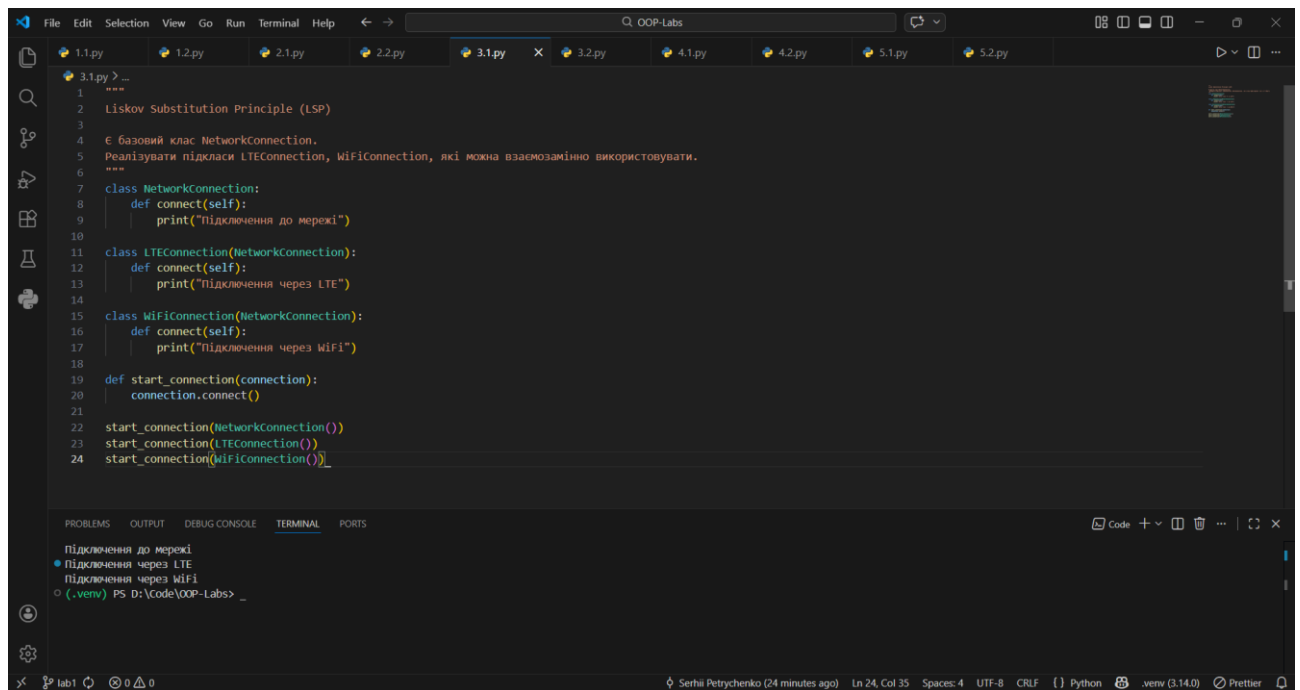


```
1 """
2 Open/Closed Principle (OCP)
3
4 Система підтримує тарифи: VoiceTariff, DataTariff.
5 Розширити систему тарифом RoamingTariff, не змінюючи логіку розрахунку вартості.
6 """
7 class Tariff:
8     def calculate_cost(self, duration):
9         pass
10
11 class VoiceTariff(Tariff):
12     def calculate_cost(self, duration):
13         return duration * 1
14
15 class DataTariff(Tariff):
16     def calculate_cost(self, duration):
17         return duration * 0.5
18
19 class RoamingTariff(Tariff):
20     def calculate_cost(self, duration):
21         return duration * 2
22
23 def calculate_call_cost(tariff, duration):
24     return tariff.calculate_cost(duration)
25
26 print(calculate_call_cost(VoiceTariff(), 10))
27 print(calculate_call_cost(DataTariff(), 10))
28 print(calculate_call_cost(RoamingTariff(), 10))
```

10
5.0
20
PS D:\Code\OOP-Labs>

Скріншот 4 – Виконання завдання 2.2

Завдання 3.1



```
File Edit Selection View Go Run Terminal Help
3.1.py x 3.2.py 4.1.py 4.2.py 5.1.py 5.2.py

1 """
2 Liskov Substitution Principle (LSP)
3
4 Є базовий клас NetworkConnection.
5 Реалізувати підкласи LTEConnection, WifiConnection, які можна взаємозаміню використовувати.
6 """
7 class NetworkConnection:
8     def connect(self):
9         print("Підключення до мережі")
10
11 class LTEConnection(NetworkConnection):
12     def connect(self):
13         print("Підключення через LTE")
14
15 class WifiConnection(NetworkConnection):
16     def connect(self):
17         print("Підключення через WiFi")
18
19 def start_connection(connection):
20     connection.connect()
21
22 start_connection(NetworkConnection())
23 start_connection(LTEConnection())
24 start_connection(WifiConnection())
```

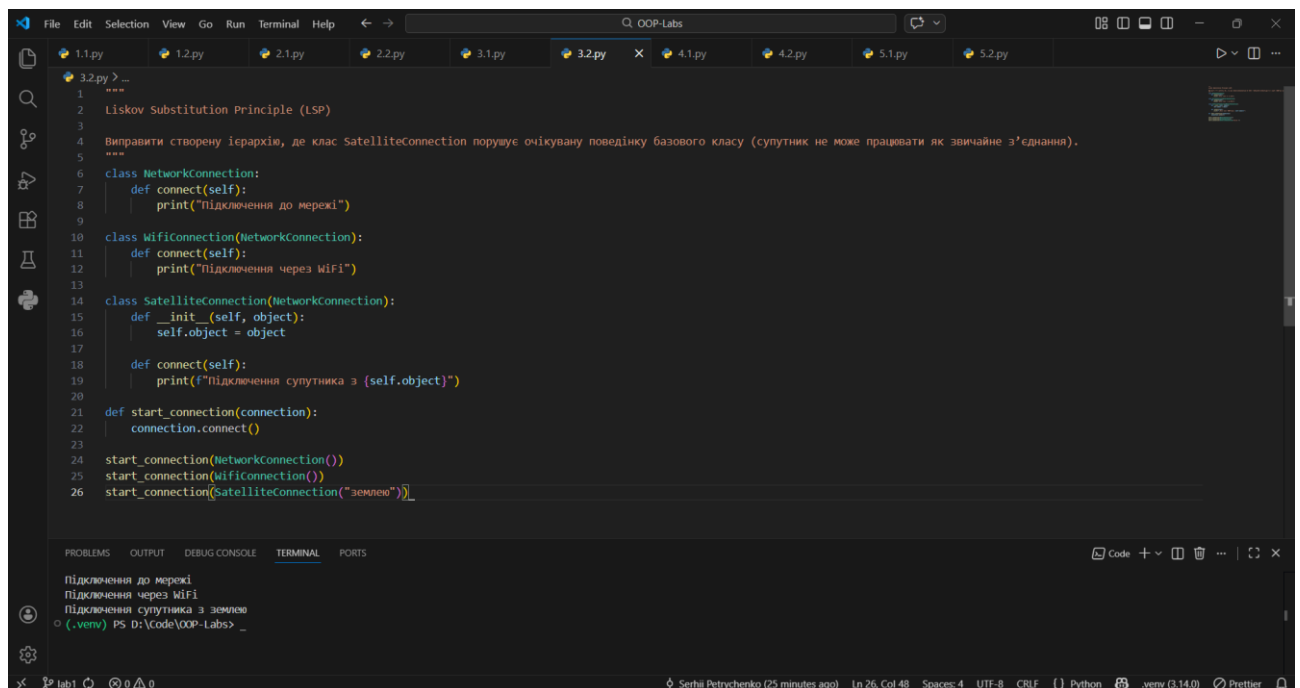
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Підключення до мережі
Підключення через LTE
Підключення через WiFi
(.venv) PS D:\Code\OOP-Labs > _

Serhii Petrychenko (24 minutes ago) Ln 24, Col 35 Spaces: 4 UTF-8 CRLF Python .venv (3.14.0) Prettier

Скріншот 5 – Виконане завдання 3.1

Завдання 3.2



```
File Edit Selection View Go Run Terminal Help
3.1.py 3.2.py x 4.1.py 4.2.py 5.1.py 5.2.py

1 """
2 Liskov Substitution Principle (LSP)
3
4 Виправити створену ієрархію, де клас SatelliteConnection порушує очікувану поведінку базового класу (супутник не може працювати як звичайне з'єднання).
5 """
6 class NetworkConnection:
7     def connect(self):
8         print("Підключення до мережі")
9
10 class WifiConnection(NetworkConnection):
11     def connect(self):
12         print("Підключення через WiFi")
13
14 class SatelliteConnection(NetworkConnection):
15     def __init__(self, object):
16         self.object = object
17
18     def connect(self):
19         print(f"Підключення супутника з {self.object}")
20
21 def start_connection(connection):
22     connection.connect()
23
24 start_connection(NetworkConnection())
25 start_connection(WifiConnection())
26 start_connection(SatelliteConnection("землю"))
```

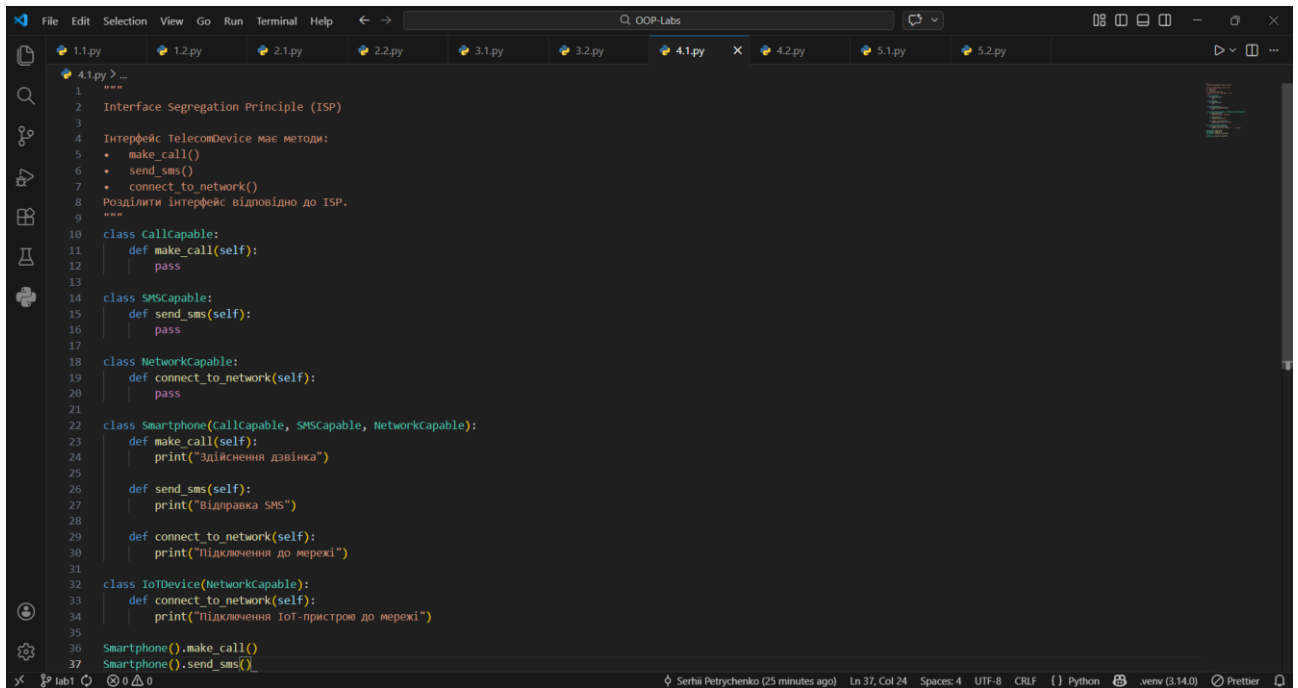
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Підключення до мережі
Підключення через WiFi
Підключення супутника з землею
(.venv) PS D:\Code\OOP-Labs > _

Serhii Petrychenko (25 minutes ago) Ln 26, Col 48 Spaces: 4 UTF-8 CRLF Python .venv (3.14.0) Prettier

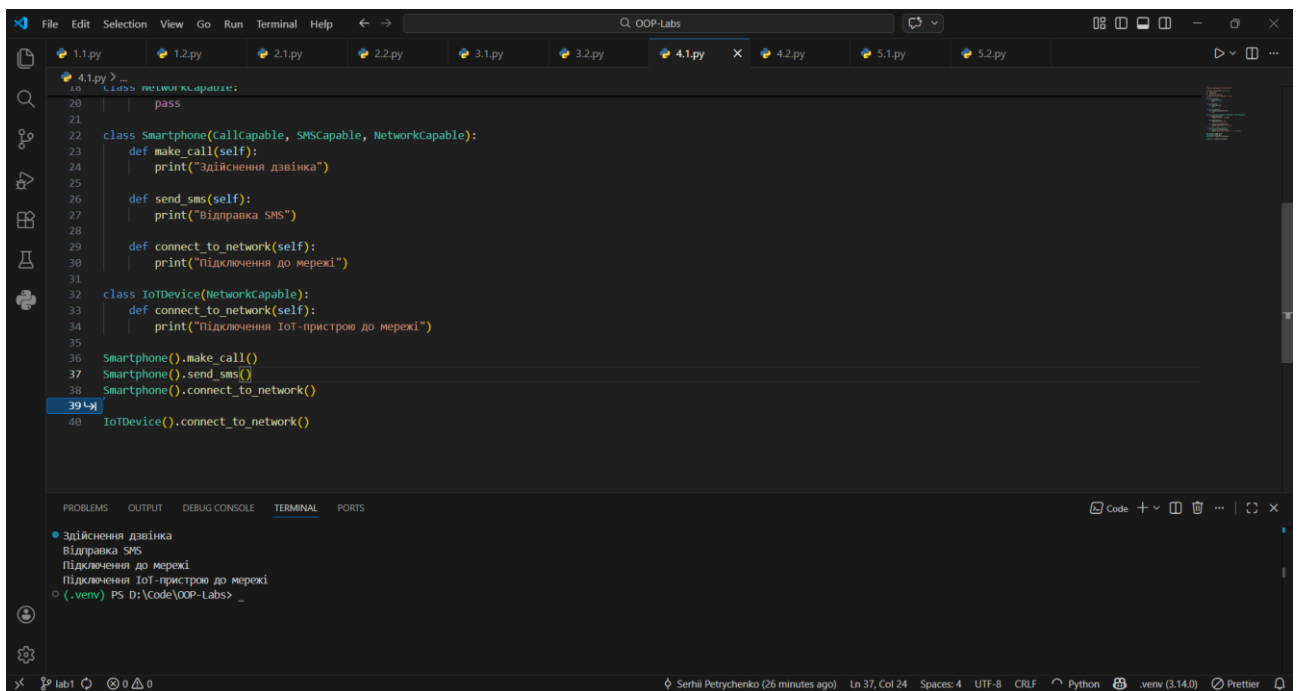
Скріншот 6 – Виконане завдання 3.2

Завдання 4.1



```
1  """
2  Interface Segregation Principle (ISP)
3
4  Інтерфейс TelecomDevice має методи:
5  • make_call()
6  • send_sms()
7  • connect_to_network()
8  Розділити інтерфейс відповідно до ISP.
9  """
10
11 class CallCapable:
12     def make_call(self):
13         pass
14
15 class SMSCapable:
16     def send_sms(self):
17         pass
18
19 class NetworkCapable:
20     def connect_to_network(self):
21         pass
22
23 class Smartphone(CallCapable, SMSCapable, NetworkCapable):
24     def make_call(self):
25         print("Здійснення дзвінка")
26
27     def send_sms(self):
28         print("Відправка SMS")
29
30     def connect_to_network(self):
31         print("Підключення до мережі")
32
33 class IoTDevice(NetworkCapable):
34     def connect_to_network(self):
35         print("Підключення IoT-пристрою до мережі")
36
37 Smartphone().make_call()
38 Smartphone().send_sms()
```

Скріншот 7 – Виконане завдання 4.1, частина 1



```
18 class TelecomDevice:
19     pass
20
21
22 class Smartphone(CallCapable, SMSCapable, NetworkCapable):
23     def make_call(self):
24         print("Здійснення дзвінка")
25
26     def send_sms(self):
27         print("Відправка SMS")
28
29     def connect_to_network(self):
30         print("Підключення до мережі")
31
32 class IoTDevice(NetworkCapable):
33     def connect_to_network(self):
34         print("Підключення IoT-пристрою до мережі")
35
36 Smartphone().make_call()
37 Smartphone().send_sms()
38 Smartphone().connect_to_network()
39 IoTDevice().connect_to_network()
```

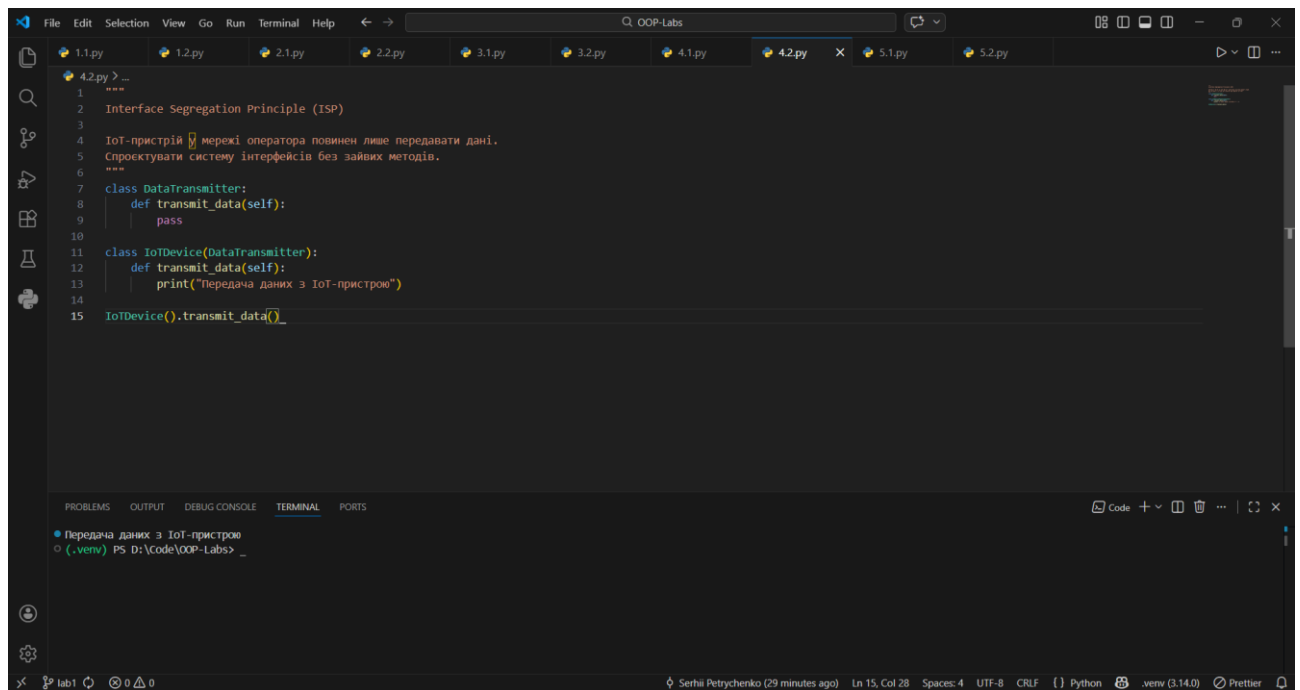
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• Здійснення дзвінка
Відправка SMS
Підключення до мережі
Підключення IoT-пристрою до мережі

(.venv) PS D:\Code\OOP-Labs> _

Скріншот 8 – Виконане завдання 4.1, частина 2

Завдання 4.2



```
File Edit Selection View Go Run Terminal Help
Q. OOP-Labs

1.1.py 1.2.py 2.1.py 2.2.py 3.1.py 3.2.py 4.1.py 4.2.py x 5.1.py 5.2.py

4.2.py > _
1 """
2 Interface Segregation Principle (ISP)
3
4 IoT-пристрій мережі оператора повинен лише передавати дані.
5 Спроектувати систему інтерфейсів без зайвих методів.
6 """
7 class DataTransmitter:
8     def transmit_data(self):
9         pass
10
11 class IoTDevice(DataTransmitter):
12     def transmit_data(self):
13         print("Передача даних з IoT-пристрою")
14
15 IoTDevice().transmit_data()
```

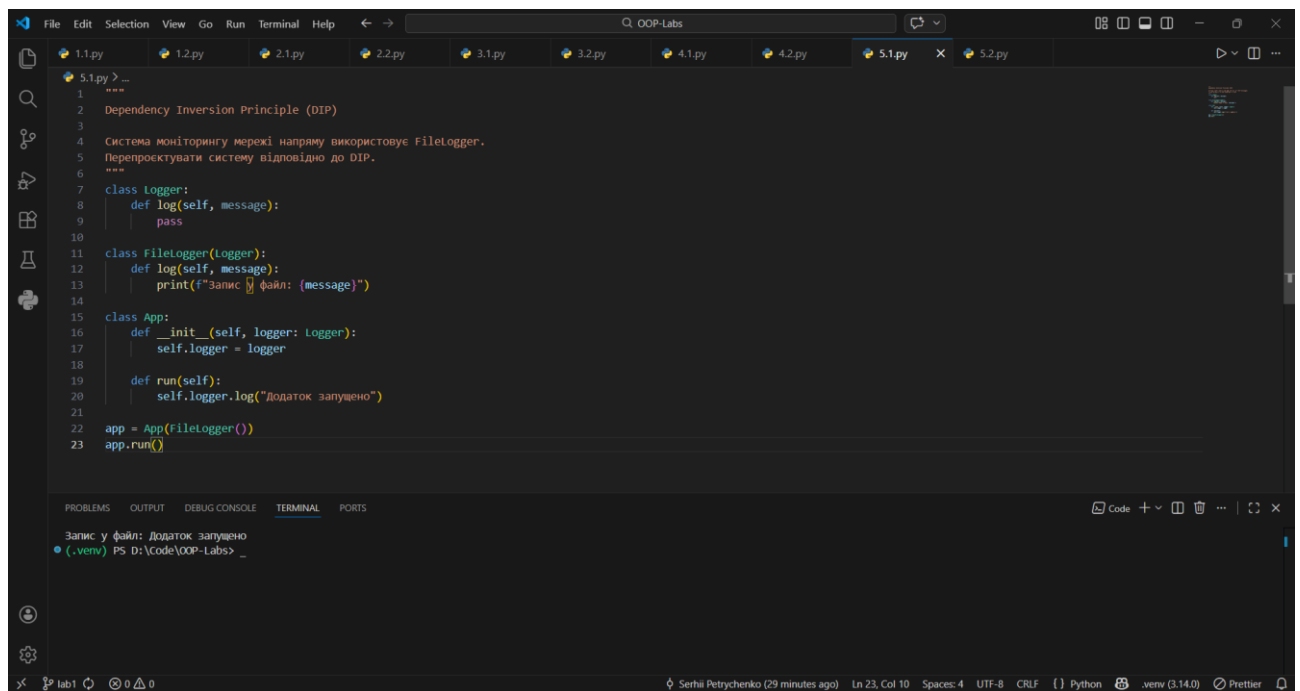
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• Передача даних з IoT-пристрою
(.venv) PS D:\Code\OOP-Labs> _

Serhii Petrychenko (29 minutes ago) Ln 15, Col 28 Spaces: 4 UTF-8 CRLF Python .venv (3.14.0) Prettier

Скріншот 9 – Виконане завдання 4.2

Завдання 5.1



```
File Edit Selection View Go Run Terminal Help
Q. OOP-Labs

1.1.py 1.2.py 2.1.py 2.2.py 3.1.py 3.2.py 4.1.py 4.2.py 5.1.py x 5.2.py

5.1.py > _
1 """
2 Dependency Inversion Principle (DIP)
3
4 Система моніторингу мережі напруги використовує FileLogger.
5 Перепроєктувати систему відповідно до DIP.
6 """
7 class Logger:
8     def log(self, message):
9         pass
10
11 class FileLogger(Logger):
12     def log(self, message):
13         print(f"Запис у файл: {message}")
14
15 class App:
16     def __init__(self, logger: Logger):
17         self.logger = logger
18
19     def run(self):
20         self.logger.log("Додаток запущено")
21
22 app = App(FileLogger())
23 app.run()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Запис у файл: Додаток запущено
(.venv) PS D:\Code\OOP-Labs> _

Serhii Petrychenko (29 minutes ago) Ln 23, Col 10 Spaces: 4 UTF-8 CRLF Python .venv (3.14.0) Prettier

Скріншот 10 – Виконане завдання 5.1

Завдання 5.2

```
1  """
2  Dependency Inversion Principle (DIP)
3
4  Реалізувати можливість підключення:
5  • FileLogger
6  • ServerLogger
7  • ConsoleLogger
8  Без зміни у коді класу NetworkMonitor.
9  """
10
11 class Logger:
12     def log(self, message):
13         pass
14
15 class FileLogger(Logger):
16     def log(self, message):
17         print(f"Запис у файл: {message}")
18
19 class ServerLogger(Logger):
20     def log(self, message):
21         print(f"Запис на сервер: {message}")
22
23 class ConsoleLogger(Logger):
24     def log(self, message):
25         print(f"Вивід на консоль: {message}")
26
27 class NetworkMonitor:
28     def __init__(self, logger: Logger):
29         self.logger = logger
30
31     def monitor(self):
32         self.logger.log("Моніторинг мережі")
33
34 monitor = NetworkMonitor(FileLogger())
35 monitor.monitor()
36 monitor = NetworkMonitor(ServerLogger())
37 monitor.monitor()
38 monitor = NetworkMonitor(ConsoleLogger())
39 monitor.monitor()
```

Скріншот 11 – Виконане завдання 5.2, частина 1

```
14 class FileLogger(Logger):
15     def log(self, message):
16         print(f"Запис у файл: {message}")
17
18 class ServerLogger(Logger):
19     def log(self, message):
20         print(f"Запис на сервер: {message}")
21
22 class ConsoleLogger(Logger):
23     def log(self, message):
24         print(f"Вивід на консоль: {message}")
25
26 class NetworkMonitor:
27     def __init__(self, logger: Logger):
28         self.logger = logger
29
30     def monitor(self):
31         self.logger.log("Моніторинг мережі")
32
33 monitor = NetworkMonitor(FileLogger())
34 monitor.monitor()
35 monitor = NetworkMonitor(ServerLogger())
36 monitor.monitor()
37 monitor = NetworkMonitor(ConsoleLogger())
38 monitor.monitor()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• Запис у файл: Моніторинг мережі
Запис на сервер: Моніторинг мережі
Вивід на консоль: Моніторинг мережі
(.venv) PS D:\code\OOP-Labs> _

Скріншот 12 – Виконане завдання 5.2, частина 2

Висновки

1. У ході роботи було вивчено архітектурні принципи SOLID та практично застосовано їх для проєктування класів на мові Python у сфері телекомунікацій. Зокрема, було виконано рефакторинг коду для забезпечення єдиної відповідальності класів, реалізовано систему тарифікації згідно з принципом відкритості/закритості та побудовано ієрархію мережових з'єднань, що відповідає принципу підстановки Лісков . Також було спроектовано систему спеціалізованих інтерфейсів для IoT-пристроїв та реалізовано інверсію залежностей у системі моніторингу мережі.
2. Отримані результати можна використовувати для створення масштабованих програмних систем, які легко модифікувати та тестувати без ризику порушити стабільність існуючого коду. У реальних телекомунікаційних проєктах ці підходи дозволяють інтегрувати нові типи вузлів та протоколів через стандартизовані інтерфейси та методи.
3. Робота допомогла сформувати навички створення гнучкого та підтримуваного коду, що є критично важливим для систем, які працюють у реальному часі. Я навчився уникати жорстких залежностей між високорівневою логікою управління та низькорівневою технічною реалізацією, що значно спрощує подальше супроводження та розширення системи.
4. Переваги: підвищення рівня повторного використання коду, спрощення процесу тестування окремих модулів та можливість швидкої адаптації системи до нових вимог бізнесу. Недоліки: необхідність ретельного попереднього проєктування архітектури та потенційне збільшення кількості дрібних класів та інтерфейсів, що на початкових етапах може дещо ускладнити структуру проєкту.